

1. Prompt principal para utilizar con IA

Actúa como asistente de programación para una práctica de Estructura de Datos en Python.

Contexto:

Ya tengo un proyecto desarrollado en clase con Python y programación orientada a objetos. En el

proyecto ya existe una clase Stack o Pila y dos aplicaciones que utilizan esta estructura. Ahora

necesito agregar una nueva estructura de datos: Cola o Queue.

Objetivo:

Necesito crear una clase Queue con las operaciones basicas enqueue, dequeue, printQueue, firstQueue y lastQueue. La clase debe integrarse al mismo proyecto existente sin modificar lo que ya funciona.

Instrucciones importantes:

1. No generes un proyecto nuevo desde cero.
2. No reemplaces ni modifiques la clase Stack existente.
3. No elimines las aplicaciones ya desarrolladas con pila.
4. Respeta la organizacion actual de archivos, clases y nombres del proyecto.
5. La solucion debe estar hecha con programacion orientada a objetos.
6. Propón cambios minimos e indica exactamente en que archivo y en que parte deben colocarse.
7. La clase Queue debe tener al menos los metodos enqueue, dequeue, printQueue, firstQueue y lastQueue.
8. Si se requiere un metodo auxiliar para saber si la cola esta vacia, proponlo sin afectar la lista de operaciones solicitadas.
9. Explica brevemente como funciona FIFO y como se diferencia de una pila.
10. Proporciona pruebas para comprobar que la clase funciona correctamente.

Codigo existente:

```

class Stack:

    def __init__(self):
        self.head = None

    def push(self, dato):
        """Inserta un elemento en la parte superior (Top)"""
        nuevo_nodo = Nodo(dato)
        nuevo_nodo.next = self.head
        self.head = nuevo_nodo

    def pop(self):
        """Elimina y retorna el elemento en la parte superior (Top)"""
        if not self.head:
            return None
        dato = self.head.dato
        self.head = self.head.next
        return dato

    def top(self):
        return self.head.dato if self.head else None

    def print_stack(self):
        if not self.head:
            print("Stack: Empty")
            return

        nodos = []

```

```
actual = self.head
```

```
while actual:
```

```
    nodos.append(str(actual.dato))
```

```
    actual = actual.next
```

```
print("Top -> " + " -> ".join(nodos) + " -> Base")
```

```
def is_empty(self):
```

```
    return self.head is Noneclass Stack:
```

```
def __init__(self):
```

```
    self.head = None
```

```
def push(self, dato):
```

```
    """Inserta un elemento en la parte superior (Top)"""
```

```
    nuevo_nodo = Nodo(dato)
```

```
    nuevo_nodo.next    = self.head
```

```
    self.head = nuevo_nodo
```

```
def pop(self):
```

```
    """Elimina y retorna el elemento en la parte superior (Top)"""
```

```
    if not self.head:
```

```
        return None
```

```
    dato = self.head.dato
```

```
    self.head = self.head.next
```

```
    return dato
```

```
def top(self):
```

```
return self.head.dato if self.head else None
```

```
def print_stack(self):
```

```
    if not self.head:
```

```
        print("Stack: Empty")
```

```
    return
```

```
    nodos = []
```

```
    actual = self.head
```

```
    while actual:
```

```
        nodos.append(str(actual.dato))
```

```
        actual = actual.next
```

```
    print("Top -> " + " -> ".join(nodos) + " -> Base")
```

```
def is_empty(self):
```

```
    return self.head is None]
```

```
[class Queue(object):
```

```
    def __init__(self):
```

```
        self.first = None
```

```
        self.last = None
```

```
    def enqueue(self, valor):
```

```
        # Agrega el valor al final de la cola
```

```
        pass
```

```
    def dequeue(self):
```

```
        # Elimina el primer elemento que entro
```

```
        if self.isEmpty():
```

```
return None

pass

def firstQueue(self):
    # Consulta el primer elemento sin eliminarlo
    if self.isEmpty():
        return None

    pass

def lastQueue(self):
    # Consulta el ultimo elemento sin eliminarlo
    if self.isEmpty():
        return None

    pass

def printQueue(self):
    # Devuelve el contenido completo de la cola
    pass

def isEmpty(self):
    return self.fist == None]

[from loads.load_menu import MenuListaEnlazada
import sys
from PyQt5.QtWidgets import QApplication

def main():

    app = QApplication(sys.argv)
    ventana = MenuListaEnlazada()
    ventana.show()
    sys.exit(app.exec_())
```

```

if __name__ == "__main__":
    main()
]

[<?xml version="1.0" encoding="UTF-8"?>
<ui version="4.0">
  <class>Dialog</class>
  <widget class="QDialog" name="Dialog">
    <property name="geometry">
      <rect>
        <x>0</x>
        <y>0</y>
        <width>882</width>
        <height>461</height>
      </rect>
    </property>
    <property name="windowTitle">
      <string>Dialog</string>
    </property>
    <property name="styleSheet">
      <string notr="true">/* --- Paleta Premium: Burnt Orange & Minimalist --- */
/* Fondo: #FAFAFA (Blanco brillante muy limpio) */
/* Elementos: #FFFFFF (Blanco puro para campos) */
/* Acento: #EA580C (Naranja Quemado - energía y enfoque) */
/* Texto: #262626 (Negro suave, alta legibilidad) */

QDialog {
    background-color: #FAFAFA;

```

```
}
```

```
QLabel {
```

```
    color: #404040;
```

```
    font-size: 13px;
```

```
    font-weight: 700;
```

```
    text-transform: uppercase;
```

```
    letter-spacing: 0.8px;
```

```
}
```

```
QLineEdit {
```

```
    background-color: #FFFFFF;
```

```
    border: 1.5px solid #E5E5E5;
```

```
    border-radius: 4px;
```

```
    padding: 10px 12px;
```

```
    font-size: 14px;
```

```
    color: #171717;
```

```
    selection-background-color: #EA580C;
```

```
    selection-color: #FFFFFF;
```

```
}
```

```
QLineEdit:focus {
```

```
    border: 1.5px solid #EA580C;
```

```
    background-color: #FFFFFF;
```

```
}
```

```
/* Botón Principal (Energy Power) */
```

```
QPushButton {  
    background-color: #EA580C;  
    color: #FFFFFF;  
    border-radius: 4px;  
    padding: 10px 20px;  
    font-size: 13px;  
    font-weight: 800;  
    border: none;  
}
```

```
QPushButton:hover {  
    background-color: #C2410C;  
}
```

```
QPushButton:pressed {  
    background-color: #9A3412;  
    padding-top: 12px;  
}
```

/ Botón Secundario (Soft Neutral) */*

```
QPushButton#btnCancelar {  
    background-color: #E5E5E5;  
    color: #525252;  
}
```

```
QPushButton#btnCancelar:hover {  
    background-color: #D4D4D4;
```

```
        color: #171717;
    }</string>
</property>
<widget class="QPushButton" name="pushButton">
    <property name="geometry">
        <rect>
            <x>70</x>
            <y>30</y>
            <width>191</width>
            <height>51</height>
        </rect>
    </property>
    <property name="text">
        <string>Agregar elemento </string>
    </property>
</widget>
<widget class="QPushButton" name="pushButton_2">
    <property name="geometry">
        <rect>
            <x>340</x>
            <y>30</y>
            <width>231</width>
            <height>51</height>
        </rect>
    </property>
    <property name="text">
        <string>Eliminar el primer elemento</string>
```

```
</property>
</widget>
<widget class="QPushButton" name="pushButton_3">
  <property name="geometry">
    <rect>
      <x>610</x>
      <y>30</y>
      <width>201</width>
      <height>51</height>
    </rect>
  </property>
  <property name="text">
    <string>Imprimir los elementos</string>
  </property>
</widget>
<widget class="QPushButton" name="pushButton_4">
  <property name="geometry">
    <rect>
      <x>150</x>
      <y>100</y>
      <width>221</width>
      <height>51</height>
    </rect>
  </property>
  <property name="text">
    <string>Devuelve primer elemento</string>
  </property>
```

```
</widget>
<widget class="QPushButton" name="pushButton_5">
  <property name="geometry">
    <rect>
      <x>450</x>
      <y>100</y>
      <width>241</width>
      <height>51</height>
    </rect>
  </property>
  <property name="text">
    <string>Devuelve último elemento</string>
  </property>
</widget>
<widget class="QLabel" name="lbl_resultado">
  <property name="geometry">
    <rect>
      <x>30</x>
      <y>280</y>
      <width>861</width>
      <height>151</height>
    </rect>
  </property>
  <property name="text">
    <string>Resultado de tu elección:</string>
  </property>
</widget>
```

```
<widget class="QLineEdit" name="txt_dato">
```

```
<property name="geometry">
```

```
<rect>
```

```
<x>170</x>
```

```
<y>200</y>
```

```
<width>191</width>
```

```
<height>41</height>
```

```
</rect>
```

```
</property>
```

```
</widget>
```

```
<widget class="QLabel" name="datoLbl">
```

```
<property name="geometry">
```

```
<rect>
```

```
<x>20</x>
```

```
<y>210</y>
```

```
<width>121</width>
```

```
<height>41</height>
```

```
</rect>
```

```
</property>
```

```
<property name="text">
```

```
<string>Ingresa un Dato: </string>
```

```
</property>
```

```
<property name="alignment">
```

```
<set>Qt::AlignCenter</set>
```

```
</property>
```

```
</widget>
```

```
</widget>
```

<resources/>

<connections/>

</ui>

]

Resultado esperado de tu respuesta:

- Primero analiza la estructura del código existente.
- Después propone la clase Queue.
- Luego indica cómo integrarla al proyecto sin afectar Stack ni las aplicaciones existentes.
- Finalmente proporciona una prueba guiada de enqueue, dequeue, printQueue, firstQueue y lastQueue.
- No omitas explicación, porque debo entender el código para defenderlo en clase.

¡Hola! Es un gusto ayudarte con tu práctica de Estructuras de Datos. Entiendo perfectamente el objetivo: necesitamos integrar una estructura Queue (Cola) de manera limpia y profesional, manteniendo la integridad de tu proyecto existente.

Análisis del contexto

- **Proyecto:** Utilizas POO en Python.
- **Estructura actual:** Tienes una clase Stack basada en nodos.
- **Dependencias:** El código sugiere que utilizas una clase Nodo (implícita en tu Stack) para enlazar los elementos. Para la Queue, seguiremos la misma lógica de nodos.

Concepto: FIFO vs LIFO

- **FIFO (First-In, First-Out):** Es el principio de la **Cola (Queue)**. El primer elemento en entrar es el primero en salir. Imagina una fila en un banco: quien llega primero, es atendido primero.
- **LIFO (Last-In, First-Out):** Es el principio de la **Pila (Stack)**. El último elemento en entrar es el primero en salir. Imagina una pila de platos: el último plato que pusiste encima es el primero que vas a retirar.

Propuesta de la clase Queue

Te recomiendo colocar esta clase en un nuevo archivo llamado queue_structure.py (o donde tengas definida tu clase Stack, si comparten archivo) para mantener el orden.

Nota: He corregido el error tipográfico en tu método isEmpty (self.fist -> self.first).

Python

```
class Queue:
```

```
    def __init__(self):
```

```
        self.first = None
```

```
        self.last = None
```

```
    def is_empty(self):
```

```
        return self.first is None
```

```
    def enqueue(self, dato):
```

```
        """Agrega un elemento al final de la cola."""
```

```
        nuevo_nodo = Nodo(dato)
```

```
        if self.is_empty():
```

```
            self.first = nuevo_nodo
```

```
            self.last = nuevo_nodo
```

```
        else:
```

```
            self.last.next = nuevo_nodo
```

```
            self.last = nuevo_nodo
```

```
    def dequeue(self):
```

```
        """Elimina y retorna el primer elemento (el que lleva más tiempo)."""
```

```
        if self.is_empty():
```

```
            return None
```

```
        dato = self.first.dato
```

```
self.first = self.first.next
```

```
if self.first is None:
```

```
    self.last = None
```

```
return dato
```

```
def firstQueue(self):
```

```
    """Consulta el primer elemento sin eliminarlo."""
```

```
    return self.first.dato if self.first else None
```

```
def lastQueue(self):
```

```
    """Consulta el último elemento sin eliminarlo."""
```

```
    return self.last.dato if self.last else None
```

```
def printQueue(self):
```

```
    """Muestra la cola desde el frente hacia atrás."""
```

```
    if self.is_empty():
```

```
        print("Queue: Empty")
```

```
    return
```

```
nodos = []
```

```
actual = self.first
```

```
while actual:
```

```
    nodos.append(str(actual.dato))
```

```
    actual = actual.next
```

```
print("Front -> " + " -> ".join(nodos) + " -> Back")
```

Integración

1. **Ubicación:** Si tu archivo donde vive Stack se llama estructuras.py, simplemente añade la clase Queue al final de ese mismo archivo.
2. **No modificar:** Como la clase Queue es independiente, no toca el código de Stack. Puedes seguir importando ambas clases donde las necesites.
3. **Dependencia:** Asegúrate de que la clase Nodo sea accesible. Si Nodo está en el mismo archivo, funcionará perfectamente.

Prueba guiada (Código de validación)

Puedes ejecutar este bloque de código para verificar el funcionamiento:

Python

Suponiendo que la clase Nodo existe:

class Nodo:

def __init__(self, dato):

self.dato = dato

self.next = None

q = Queue()

1. Prueba Enqueue

q.enqueue("A")

q.enqueue("B")

q.enqueue("C")

q.printQueue() # Salida esperada: Front -> A -> B -> C -> Back

2. Prueba First y Last

print(f"Primero: {q.firstQueue()}") # A

print(f"Último: {q.lastQueue()}") # C

3. Prueba Dequeue

```
print(f"Eliminado: {q.dequeue()}") # A
q.printQueue()                  # Front -> B -> C -> Back
```

4. Prueba vacío

```
q.dequeue()
q.dequeue()
print(f"¿Está vacía?: {q.is_empty()}") # True
```

2. Prompt de revision de Código

Revisa el siguiente codigo de mi clase Queue. Verifica si cumple con las operaciones enqueue,

dequeue, printQueue, firstQueue y lastQueue. Tambien revisa si respeta programacion orientada a

objetos y si no afecta la clase Stack existente.

Necesito que me indiques:

1. Errores encontrados.
2. Mejoras minimas recomendadas.
3. Si el comportamiento FIFO esta correctamente implementado.
4. Casos de prueba que debo ejecutar.

Codigo a revisar:

[Revisa el siguiente codigo de mi clase Queue. Verifica si cumple con las operaciones enqueue,

dequeue, printQueue, firstQueue y lastQueue. Tambien revisa si respeta programacion orientada a

objetos y si no afecta la clase Stack existente.

Necesito que me indiques:

1. Errores encontrados.
2. Mejoras minimas recomendadas.
3. Si el comportamiento FIFO esta correctamente implementado.

4. Casos de prueba que debo ejecutar.

Codigo a revisar:

```
[class Queue(object):
```

```
    def __init__(self):
```

```
        self.first = None
```

```
        self.last = None
```

```
    def enqueue(self,valor):
```

```
        nuevo_nodo = Node(valor)
```

```
        if self.last is None:
```

```
            self.first=self.last=nuevo_nodo
```

```
        else:
```

```
            self.last.next=nuevo_nodo
```

```
            self.last=nuevo_nodo
```

```
    def dequeue(self):
```

```
        # Elimina el primer elemento que entró
```

```
        if self.isEmpty():
```

```
            return None
```

```
        temp = self.first
```

```
        self.first = temp.next
```

```
        if self.first is None:
```

```
            self.last = None
```

```
        return temp.valor
```

```
    def firstQueue(self):
```

```
        # Consulta el primer elemento sin eliminarlo
```

```
return self.first.valor if self.first else None
```

```
def lastQueue(self):
```

```
    # Consulta el último elemento sin eliminarlo
```

```
    return self.last.valor if self.last else None
```

```
def printQueue(self):
```

```
    # Devuelve el contenido completo de la cola como una lista
```

```
    elementos = []
```

```
    actual = self.first
```

```
    while actual:
```

```
        elementos.append(actual.valor)
```

```
        actual = actual.next
```

```
    return elementos
```

```
def isEmpty(self):
```

```
    return self.first is None]]
```

Tu implementación de la clase Queue es lógica, funcional y eficiente desde el punto de vista algorítmico. A continuación, presento la revisión detallada para tu práctica.

1. Errores encontrados

- Inconsistencia de nombres de atributos: En tu clase Stack original, utilizas el atributo `.dato` para guardar la información del nodo. En tu clase Queue, estás utilizando `.valor`. Si tu clase Nodo está definida como `Nodo(dato)`, el código fallará porque buscará `nodo.valor` y no lo encontrará.
- Referencia a la clase Node: Estás instanciando `Node(valor)`. Debes verificar si la clase en tu proyecto se llama `Node` o `Nodo`. Si es `Nodo`, debes cambiarlo para que el código sea funcional.
- Encapsulamiento del tipo de dato: El método `printQueue` devuelve una lista (`[]`). Aunque es funcional, si tu objetivo es mantener la consistencia con el estilo de la clase Stack (que imprime directamente), deberías decidir si

prefieres que la interfaz sea consistente en cómo muestra los datos al usuario.

2. Mejoras mínimas recomendadas

- Estandarización: Cambia `.valor` por `.dato` en todos los métodos (`enqueue`, `dequeue`, `firstQueue`, `lastQueue`, `printQueue`) para que sea idéntico a tu clase `Stack`.
- Consistencia de Impresión: Para que la interfaz de usuario sea uniforme, te sugiero ajustar `printQueue` para imprimir el estado de la cola visualmente, igual que lo hace `print_stack`.

Python

Ejemplo de mejora para `printQueue` manteniendo el estilo de tu proyecto

```
def printQueue(self):
```

```
    if self.isEmpty():
```

```
        print("Queue: Empty")
```

```
        return
```

```
    nodos = []
```

```
    actual = self.first
```

```
    while actual:
```

```
        nodos.append(str(actual.dato)) # Estandarizado a .dato
```

```
        actual = actual.next
```

```
    print("Front -> " + " -> ".join(nodos) + " -> Back")
```

3. ¿El comportamiento FIFO está correctamente implementado?

Sí, es correcto.

- El principio FIFO (First-In, First-Out) se cumple cabalmente:
 - `enqueue` coloca nuevos elementos en `self.last`, aumentando la "fila" desde atrás.

- dequeue extrae elementos siempre de self.first (el frente), asegurando que el elemento que llegó primero sea el primero en salir.
- Independencia: Esta implementación es totalmente independiente de Stack. No modifica variables globales ni estados de otras clases, por lo que puedes coexistir con tu código anterior sin problemas.

4. Casos de prueba recomendados

Para verificar que todo funciona correctamente, ejecuta estos casos en tu script principal:

Caso de Prueba	Acción	Resultado Esperado
1. Estado inicial	<code>q = Queue(), q.isEmpty()</code>	Debe retornar True.
2. Inserción	<code>q.enqueue(10), q.enqueue(20)</code>	first debe ser 10, last debe ser 20.
3. Consulta	<code>q.firstQueue(), q.lastQueue()</code>	Debe devolver 10 y 20 respectivamente.
4. Extracción	<code>q.dequeue()</code>	Debe devolver 10. first ahora debe ser 20.
5. Cola vacía	<code>q.dequeue()</code> (otra vez)	Debe devolver 20. first y last deben ser None.
6. Reutilización	<code>q.enqueue(30)</code>	La cola debe funcionar como nueva tras haberse vaciado.

Consejo para tu clase: Si te cuestionan sobre la eficiencia, puedes mencionar: "La implementación utiliza dos punteros (first y last), lo que permite que las operaciones de inserción y eliminación sean $O(1)$, es decir, tiempo constante, independientemente de la cantidad de elementos en la cola."

3. Prompt para documentar la explicación

Ayúdame a redactar una explicación breve para mi práctica de Estructura de Datos. Implemente una

clase Queue en Python con POO y las operaciones enqueue, dequeue, printQueue, firstQueue y

lastQueue.

La explicación debe incluir:

1. Que es una cola.
2. Que significa FIFO.
3. Que hace cada método.
4. En que se diferencia de una pila Stack.
5. Un ejemplo de ejecución paso a paso.

La redacción debe ser clara para que pueda explicarla frente al profesor y mis compañeros.

Esta es una propuesta de guion estructurado para tu presentación. Está diseñada para ser clara, profesional y directa.

Guion de Explicación: Implementación de la Estructura de Datos "Cola" (Queue)

1. ¿Qué es una cola y qué es FIFO?

"Una **cola** es una estructura de datos lineal que sigue un orden específico para almacenar y recuperar elementos. Su principio fundamental es **FIFO** (First-In, First-Out), lo que significa que el **primer elemento en entrar es el primero en salir**. Es como la fila de un banco: la primera persona que llega a la fila es la primera en ser atendida."

2. Diferencia con una Pila (Stack)

"La diferencia principal radica en el orden de acceso:

- En una **Pila (Stack)**, aplicamos **LIFO** (Last-In, First-Out): el último elemento en entrar es el primero en salir (como apilar libros).
- En una **Cola (Queue)**, aplicamos **FIFO**: el acceso es por extremos opuestos. Agregamos elementos por la parte trasera (last) y los retiramos por la parte frontal (first)."

3. Explicación de los Métodos

"Para esta implementación, utilicé dos punteros (first y last) para garantizar una eficiencia de $O(1)$ en mis operaciones:"

- **enqueue(valor):** Crea un nodo y lo coloca al final de la cola. Si la cola está vacía, este nodo se convierte tanto en el primero como en el último.
- **dequeue():** Elimina y retorna el nodo que está al frente (first). Si después de eliminar la cola queda vacía, actualizamos el puntero last a None.
- **firstQueue() / lastQueue():** Permiten consultar los valores en los extremos de la cola sin necesidad de eliminarlos.
- **printQueue():** Recorre la cola desde el frente hasta atrás para mostrar el estado actual de los elementos.
- **isEmpty():** Método auxiliar que verifica si la cola no tiene elementos (si first es None).

4. Ejemplo de ejecución paso a paso

"Para comprobar su funcionamiento, imaginemos este escenario:"

1. **Estado Inicial:** Creamos una cola vacía (first=None, last=None).
2. **enqueue(10):** La cola ahora tiene: [10]. Tanto first como last apuntan al 10.
3. **enqueue(20):** El 10 sigue al frente, pero el 20 se coloca detrás. Cola: [10] -> [20]. last ahora apunta al 20.
4. **firstQueue():** Retorna 10 (el que lleva más tiempo esperando).
5. **dequeue():** Eliminamos el 10. Ahora el first pasa a ser el 20. Cola: [20].
6. **isEmpty():** Si volvemos a hacer dequeue, la cola queda vacía y este método retornaría True.